Änderungsprotokoll — Adressverwaltung v2.5.0

Datum: 10. Oktober 2026
Ausgangsstand: v2.4.0 (Commit `8f234fe`)
Endstand: Commits `b1d6d09`, `6e20e3c`, `3491199`, `9332d24`, `61c60ac`, `ea60944`, `4c8d167`, `c2c0eb2` und der Commit mit dem Sperrkennzeichen
Versionsnummer: 2.5.0 (vorher 2.4.0)

---

1. Übersicht

Version 2.5.0 fasst die Änderungen vom 10. Oktober 2026 zusammen. Die Abschnitte 2 und 3 standen zuerst als Nachträge im Änderungsprotokoll der Version 2.4.0 und sind von dort hierher verschoben.

#	Paket	Inhalt	Commit
1	Benutzerverwaltung	Neue Seite «Benutzer» für die Rolle `Admin`; neue Benutzer erhalten eine E-Mail; ein gelöschter Benutzer wird der Benachrichtigungsadresse gemeldet	`61c60ac`, `4c8d167`, `c2c0eb2`
2	Anmeldesperre	Nach 3 Fehlversuchen für 5 Minuten (bisher 5 Fehlversuche, 15 Minuten); Meldung im Login-Formular; Sperrkennzeichen am Benutzer und E-Mail bei einer Sperre	`61c60ac`, Commit mit der Fassung dieses Dokuments, die das Sperrkennzeichen beschreibt
3	Einstellungen	Nach dem Speichern wechselt die Seite zur Adressliste	`b1d6d09`, `6e20e3c`, `9332d24`
4	E-Mail-Versand	SMTP-Server und Absender eingetragen, Versand geprüft	`3491199`
5	Version	Versionsnummer auf 2.5.0 gesetzt, Tutorial umbenannt	`ea60944`
6	Dokumentation	Neues Sequenzdiagramm zum Login mit Sperrmechanismus und Schulungsunterlage dazu	Commit mit der Fassung dieses Dokuments, die Abschnitt 6 enthält

Das Datenbankschema ändert sich: Die Tabelle `Users` erhält die Spalte `LockedUntil`. Das Backend ergänzt sie beim Start selbst. Nach dem Aktualisieren: `docker compose up --build -d`, danach `docker compose restart nginx`.

---

2. Benutzerverwaltung und Anmeldesperre

Neu: Seite «Benutzer»

In der Navigation steht neben «Einstellungen» der neue Punkt «Benutzer». Ein Benutzer mit der Rolle `Admin` kann dort:

1. alle Benutzer mit Name, E-Mail-Adresse, Rolle und Status sehen,
2. einen Benutzer anlegen (E-Mail-Adresse, Anzeigename, Rolle, Passwort mit mindestens 8 Zeichen) – der neue Benutzer erhält eine E-Mail an seine Adresse,
3. Anzeigename, Rolle und Passwort ändern – ein neues Passwort meldet den Benutzer überall ab,
4. eine Anmeldesperre aufheben,
5. einen Benutzer löschen – seine Sitzung endet mit dem nächsten Aufruf, und die Benachrichtigungsadresse aus den Einstellungen erhält eine E-Mail.

Die E-Mail-Adresse lässt sich nicht ändern. Sich selbst kann ein Admin weder löschen noch die Rolle entziehen; so bleibt immer ein Admin übrig. Wer nicht `Admin` ist, sieht auf der Seite nur einen Hinweis.

Das entscheidet das Backend (Antwort 403), nicht die Rolle in der Session.

Neu: E-Mail an neue Benutzer

Wer neu angelegt wird, erhält eine E-Mail an die eigene Adresse: Anrede mit dem Anzeigenamen, die E-Mail-Adresse für die Anmeldung und die Rolle. Das Passwort steht nicht darin; es ist auf anderem Weg weiterzugeben. Das gilt für die Seite «Benutzer» (`POST /users`) und für `POST /auth/register`, nicht für `scripts/create_user.sh`. Schlägt der Versand fehl, ist der Benutzer trotzdem angelegt; der Fehler steht im Log des Backends.

Eine Mail kommt nur an, wenn die E-Mail-Adresse stimmt. Sie lässt sich nachträglich nicht ändern: Bei einem Tippfehler den Benutzer löschen und neu anlegen.

Neu: E-Mail bei gelöschtem Benutzer

Wird ein Benutzer gelöscht, geht eine E-Mail an die Benachrichtigungsadresse aus den Einstellungen – dieselbe Adresse, die auch neue Adressen gemeldet bekommt. Sie nennt Name, E-Mail-Adresse und Rolle des gelöschten Benutzers und wer ihn gelöscht hat. Ist in den Einstellungen keine Adresse hinterlegt, wird nichts gesendet. Schlägt der Versand fehl, ist der Benutzer trotzdem gelöscht; der Fehler steht im Log des Backends. Die Einstellungsseite nennt die neue Benachrichtigung in ihrem Hinweistext.

Geändert: Anmeldesperre

Nach 3 fehlerhaften Anmeldungen ist eine E-Mail-Adresse für 5 Minuten gesperrt (bisher: nach 5 Fehlversuchen für 15 Minuten). Die 5 Minuten zählen ab dem dritten Fehlversuch; bisher lief die Frist ab dem ersten. Das Login-Formular zeigt während der Sperre eine eigene Meldung statt «Ungültige E-Mail-Adresse oder falsches Passwort». Das Änderungsprotokoll der Version 2.4.0 nennt noch die alten Werte.

Unverändert: Gesperrt wird die E-Mail-Adresse, auch wenn es dazu keinen Benutzer gibt. Der Fehlerzähler liegt im Arbeitsspeicher des Backends.

Neu: Sperrkennzeichen am Benutzer und E-Mail bei einer Sperre

Gehört die gesperrte E-Mail-Adresse zu einem Benutzer, geschieht beim dritten Fehlversuch zusätzlich:

1. Beim Benutzer wird das Sperrkennzeichen gesetzt: Die neue Spalte `Users.LockedUntil` enthält das Ende der Sperre.
2. Die Benachrichtigungsadresse aus den Einstellungen erhält eine E-Mail mit Name, E-Mail-Adresse und Rolle des gesperrten Benutzers. Ist dort keine Adresse hinterlegt, wird nichts gesendet. Pro Sperre geht eine E-Mail hinaus, nicht eine pro weiterem Versuch.

Das Kennzeichen lässt sich vor Ablauf der 5 Minuten von Hand löschen: auf der Seite «Benutzer» mit «Entsperren». Die Liste zeigt es als «Gesperrt bis …». Auch ein neues Passwort löscht es. Nach Ablauf sperrt es nicht mehr; die nächste erfolgreiche Anmeldung leert das Feld.

Weil das Kennzeichen in der Datenbank steht, übersteht die Sperre eines Benutzers einen Neustart des Backends. Für eine Adresse ohne Benutzer gibt es weder Kennzeichen noch E-Mail; ihre Sperre liegt nur im Arbeitsspeicher.

Das Datenbankschema ändert sich damit: Das Backend ergänzt die Spalte `Users.LockedUntil` beim Start selbst, bestehende Daten bleiben erhalten.

Backend

Datei	Änderung
neu `Controllers/UsersController.cs`, `Dtos/UserDtos.cs`	`/users` nur für `Admin`: lesen, anlegen (409 bei vorhandener E-Mail-Adresse), ändern, löschen, `POST /users/{id}/unlock`. Antworten mit `lockedUntil` und

Datei	Änderung
	`isSelf`, ohne Passwort-Hash
`Services/ILoginThrottle.cs`, `Services/MemoryLoginThrottle.cs`	3 Fehlversuche, 5 Minuten Sperre ab dem dritten; `RegisterFailure` meldet zurück, wenn der Versuch die Sperre auslöst
`Models/User.cs`, `Data/AdresseDbContext.cs`, `SchemaUpgrader.cs`	Sperrkennzeichen `LockedUntil`; Spalte wird in einer bestehenden Datenbank ergänzt
`Services/INotificationService.cs`, `Services/EmailNotificationService.cs`	`NotifyNewUserAsync`: E-Mail an den neuen Benutzer, ohne Passwort. `NotifyUserDeletedAsync` und `NotifyUserLockedAsync`: Meldung an die Adresse aus den Einstellungen
`Controllers/AuthController.cs`	`POST /auth/register` sendet die E-Mail ebenfalls. Login setzt und prüft das Sperrkennzeichen und meldet die Sperre
`Models/Roles.cs`	Kommentar

Frontend

Datei	Änderung
neu `app/benutzer/page.tsx`, `app/benutzer/neu/page.tsx`, `app/benutzer/[id]/bearbeiten/page.tsx`	Liste, Anlegen, Bearbeiten
neu `components/BenutzerForm.tsx`, `types/benutzer.ts`	Formular und Typen; Hinweis auf die E-Mail beim Anlegen
`components/NavBar.tsx`	Link «Benutzer»
`lib/api.ts`	Funktionen für `/users`; `apiRequest` wirft neu einen `ApiFehler` mit Status und Meldung des Backends
`app/api/backend/[...pfad]/route.ts`	Bereich `users` über den Proxy erreichbar
neu `lib/anmeldung.ts`, `lib/auth.ts`, `app/login/LoginForm.tsx`	Meldung im Login-Formular, wenn die Adresse gesperrt ist

Infrastruktur und Dokumentation

Datei	Änderung
`nginx/nginx.conf`	`location /users` zum Backend
`scripts/reset_password.sh`	Löscht auch das Sperrkennzeichen
neu `AdressverwaltungApi.Tests/UsersTests.cs`	34 Tests: Rechte, Anlegen, E-Mail an neue Benutzer, Ändern, Passwortwechsel, Löschen, E-Mail bei gelöschtem Benutzer, Sperre mit Sperrkennzeichen und E-Mail, Entsperren
`documentation/Tutorial_WebApp_v2.5.html` und `.pdf`	Abschnitt «Benutzerverwaltung», Anmeldesperre, geänderte Codebeispiele (auch die SMTP-Angaben aus `docker-compose.yml`)
`documentation/Sequenzdiagramm.puml`, `Systemarchitektur.puml`, je mit `.svg`	Sperre 3 / 5 Minuten; `UsersController`
`documentation/Schulungsunterlage_Sequenzdiagramm.html`, `.pdf`, PNG zum Login	Sperre 3 / 5 Minuten
`documentation/Sicherheitsbericht.html` und `.pdf`	Angaben zur Kontosperre (S-22)
`README.md`, `CLAUDE.md`	Seite «Benutzer», Endpunkte `/users`, Sperre

Nach dem Aktualisieren zu tun

`docker compose up --build -d`, danach `docker compose restart nginx`. Die neue Spalte `Users.LockedUntil` entsteht beim Start des Backends.

Prüfung

Prüfung	Ergebnis
`dotnet test`	111 von 111 bestanden (34 neu)
Laufender Stack über die API: drei Fehlversuche, Neustart des Backends, Entsperren	Sperrkennzeichen gesetzt, E-Mail «Benutzer gesperrt» gesendet; nach dem Neustart weiterhin 429; nach dem Entsperren Kennzeichen leer und Anmeldung möglich
Typprüfung und Lint des Frontends	erfolgreich
Browser (Chromium, ferngesteuert) als `Admin`: Benutzer anlegen, doppelte E-Mail-Adresse, ändern, löschen	erfolgreich; Meldung des Backends bei doppelter Adresse sichtbar
Browser: drei Fehlversuche, danach richtiges Passwort	Meldung zur Sperre; Liste zeigt «Gesperrt bis …»; nach «Entsperren» gelingt die Anmeldung
Browser als `User`	Seite «Benutzer» zeigt nur den Hinweis
Browser: gelöschter Benutzer mit offener Sitzung	landet beim nächsten Aufruf auf der Login-Seite

Nicht geprüft: der Ablauf der Sperre nach 5 Minuten im Browser (durch den Test der Dauer im Backend abgedeckt) und andere Browser als Chromium.

---

3. Einstellungen: nach dem Speichern zur Adressliste

Nach dem Speichern der Einstellungen wechselt die Seite zur Adressliste (`/`), wie es die Formulare für Adressen und Städte nach dem Speichern auch tun. Die Meldung «Einstellungen wurden gespeichert» entfällt damit. Schlägt das Speichern fehl, bleibt die Seite stehen und zeigt den Fehler.

Datei	Änderung
`frontend/app/einstellungen/page.tsx`	`router.push('/')` nach dem Speichern; Erfolgsmeldung entfernt

Prüfung	Ergebnis
Typprüfung und Lint des Frontends	erfolgreich
Browser (Chromium, ferngesteuert) als `Admin`: Farbe ändern und speichern	Wechsel zur Adressliste, neue Farbe gilt sofort
Browser: Speichern schlägt fehl (Antwort 500 nachgestellt)	Seite bleibt stehen, Fehlermeldung sichtbar

---

4. E-Mail-Versand

In `docker-compose.yml` stehen jetzt der SMTP-Server `asmtp.mail.hostpoint.ch` und die Absenderadresse. Benutzername und Passwort kommen weiterhin aus `.env` (`SMTP_USERNAME`, `SMTP_PASSWORD`).

Der Port bleibt 587 (STARTTLS). Port 465 verlangt TLS ab Verbindungsbeginn, und das unterstützt `System.Net.Mail.SmtpClient` nicht, mit dem `EmailService` sendet.

Prüfung	Ergebnis
Verbindung zum SMTP-Server auf Port 587 mit STARTTLS	TLS 1.3, Zertifikat gültig
Neue Adresse über die API angelegt, Empfänger in den Einstellungen hinterlegt	Backend meldet «E-Mail gesendet»; der Server hat die Nachricht angenommen

Nicht geprüft: die Zustellung im Postfach.

---

5. Versionsnummer

Die Version 2.5.0 steht in `README.md`, `CLAUDE.md`, `frontend/package.json`, `frontend/package-lock.json`, in beiden Diagrammen, in der Schulungsunterlage, im Sicherheitsbericht und im Tutorial.

Datei	Änderung
`documentation/Tutorial_WebApp_v2.5.html` und `.pdf`	Umbenannt (vorher `Tutorial_WebApp_v2.4.*`); Titelseite
neu `documentation/Aenderungsprotokoll_v2.5.0.md` und `.pdf`	Dieses Dokument
`documentation/Aenderungsprotokoll_v2.4.0.md` und `.pdf`	Nachträge vom 10. Oktober 2026 in dieses Dokument verschoben

---

6. Sequenzdiagramm und Schulungsunterlage zum Sperrmechanismus

Ein eigenes Sequenzdiagramm zeigt den Login mit dem Sperrmechanismus in 64 nummerierten Schritten und fünf Abschnitten:

Abschnitt	Schritte	Inhalt
A	1 bis 14	Erster und zweiter Fehlversuch
B	15 bis 25	Dritter Fehlversuch: Sperre, Sperrkennzeichen, E-Mail
C	26 bis 35	Anmeldeversuch während der Sperre, auch nach einem Neustart des Backends
D	36 bis 49	Ende der Sperre: Entsperren durch einen Administrator oder Ablauf der 5 Minuten
E	50 bis 64	Erfolgreiche Anmeldung nach der Sperre

Die Schulungsunterlage erklärt jeden Schritt mit «Was passiert», «Im Code» und «Warum so» und enthält dazu eine Übersicht der Zustände einer Sperre, Kontrollfragen und ein Glossar.

Datei	Inhalt
neu `documentation/Sequenzdiagramm_Login_Sperre.puml` und `.svg`	Das Diagramm
neu `documentation/Schulungsunterlage_Login_Sperre.html` und `.pdf`	Erläuterung der 64 Schritte, 20 Seiten
neu `documentation/Schulungsunterlage_Login_Sperre_A.png` bis `_E.png`	Die fünf Abschnitte des Diagramms für die Schulungsunterlage
`README.md`, `CLAUDE.md`, `documentation/Tutorial_WebApp_v2.5.html` und `.pdf`	Verweis auf die neuen Dokumente

Am Code ändert sich nichts.

---

7. Nachtrag vom 11. Oktober 2026: Feld «E-Mail» an der Adresse

Eine Adresse hat neu ein siebtes Fachfeld `email`. Es ist freiwillig; Adressen, die es schon gab, bleiben ohne E-Mail-Adresse. Eine E-Mail-Adresse darf nur bei einer einzigen Adresse stehen, ohne Rücksicht auf Gross- und Kleinschreibung; Adressen ohne E-Mail-Adresse gibt es beliebig viele. Die Versionsnummer bleibt 2.5.0.

Bereich	Änderung
Datenbank	Neue Spalte `Adressen.Email` (`character varying(256)`, leer erlaubt) mit dem eindeutigen Index `IX_Adressen_Email`. Das Backend ergänzt beides beim Start selbst (`SchemaUpgrader`)

Bereich	Änderung
API	`email` in `/odata/Adressen` lesen, setzen (`POST`, `PATCH`) und mit `null` entfernen; als Feld in `$filter`, `$orderby` und `$select` verwendbar. Ein Wert, der keine E-Mail-Adresse ist, oder mehr als 256 Zeichen: 400. Gespeichert wird die Adresse ohne Leerzeichen am Rand und in Kleinbuchstaben. Steht sie schon bei einer anderen Adresse: 409 mit einer Meldung
Formular	Neues Eingabefeld «E-Mail» ohne Pflichtstern; der Browser prüft die Form der Adresse. Ein leeres Feld wird als `null` gesendet. Die Meldung des Backends zu einer bereits erfassten E-Mail-Adresse erscheint im Formular
Adressliste	Neue Spalte «E-Mail»; die Adresse ist ein `mailto:`-Link
Benachrichtigung	Die E-Mail «Neue Adresse erfasst» nennt zusätzlich die E-Mail-Adresse («–», wenn keine erfasst ist)

Datei	Änderung
`backend/AdressverwaltungApi/Models/Adresse.cs`	Eigenschaft `Email` mit `[MaxLength(256)]` und `[EmailAddress]`
`backend/AdressverwaltungApi/Data/AdresseDbContext.cs`	Eindeutiger Index auf `Email`
`backend/AdressverwaltungApi/SchemaUpgrader.cs`	Spalte `Email` und den Index in einer bestehenden Datenbank nachziehen
`backend/AdressverwaltungApi/Controllers/ODataCrudController.cs`	Neuer Erweiterungspunkt `OnSavingAsync` vor dem Speichern in `Post` und `Patch`; eine Meldung führt zu 409
`backend/AdressverwaltungApi/Controllers/AdressenController.cs`	`OnSavingAsync`: E-Mail-Adresse in Kleinbuchstaben, Prüfung auf eine andere Adresse mit derselben E-Mail-Adresse
`backend/AdressverwaltungApi/Services/EmailNotificationService.cs`	Zeile «E-Mail» in der Benachrichtigung
`frontend/types/adresse.ts`, `frontend/lib/api.ts`	Feld `email` im Typ und in `normalizeAdresse`
`frontend/components/AdresseForm.tsx`	Eingabefeld «E-Mail»; zeigt die Meldung des Backends
`frontend/app/page.tsx`	Spalte «E-Mail» in der Adressliste
`backend/AdressverwaltungApi.Tests/`	11 neue Tests (122 statt 111)
`CLAUDE.md`, `documentation/Tutorial_WebApp_v2.5.html` und `.pdf`	Feld nachgeführt

Geprüft: alle 122 Backend-Tests; im Browser (Chromium ohne Oberfläche) das Formular «Neue Adresse», das Setzen und Entfernen der E-Mail-Adresse über «Bearbeiten», die Anzeige in der Adressliste und die Meldung bei einer bereits erfassten E-Mail-Adresse; die neue Spalte und der Index in der bestehenden Datenbank.

Nicht geprüft: das Anlegen einer Adresse mit E-Mail-Adresse über die Oberfläche (nur über die Tests) und die Benachrichtigung mit der neuen Zeile im Postfach.

Enthält eine bestehende Datenbank dieselbe E-Mail-Adresse bei zwei Adressen, lässt sich der Index nicht anlegen und das Backend startet nicht; die doppelten Werte sind dann zuerst zu bereinigen.

Nach dem Aktualisieren: `docker compose up --build -d`, danach `docker compose restart nginx`.

---

8. Nachtrag vom 11. Oktober 2026: Adresse der App einstellbar, Dokumentation nachgeführt

Die Anwendung wurde erstmals auf einem Server im lokalen Netz in Betrieb genommen. Dabei zeigte sich, dass die Adresse `https://localhost` fest in `docker-compose.yml` stand: Nach der Anmeldung leitete NextAuth auf `localhost` um. Die Adresse ist jetzt über die neue, freiwillige Variable `APP_URL` in

`.env` einstellbar. Die Versionsnummer bleibt 2.5.0.

Bereich	Änderung	Commit
Konfiguration	Neue Variable `APP_URL` in `.env`: Adresse, unter der die App im Browser aufgerufen wird. Sie setzt `NEXTAUTH_URL` und `NEXT_PUBLIC_API_URL` des Frontends. Ohne Wert bleibt es bei `https://localhost`; das Zertifikat in `nginx/ssl` muss auf die Adresse lauten	`e42dc3a`
Hilfsskripte	`create_user.sh` und `reset_password.sh` nennen am Schluss die Login-Adresse aus `APP_URL` statt fest `https://localhost/login`	`209042a`
Sequenzdiagramm	Im Ablauf «Neue Adresse erfassen» fehlte die Prüfung der eindeutigen E-Mail-Adresse aus Abschnitt 7. Neu sind die Schritte 29 bis 31 (Kleinbuchstaben, Abfrage, Antwort) mit einer Notiz zum 409; das Diagramm hat 47 statt 44 Schritte	`7774a6d`
Schulungsunterlage	Schritte 29 bis 31 beschrieben, die folgenden Schritte und alle Verweise um drei verschoben; Texte zum Feld «E-Mail» und zum Bereich `/users` nachgeführt; 15 statt 14 Seiten	`7774a6d`
Tutorial	`docker-compose.yml` und `.env.example` mit `APP_URL` zitiert; Schrittzahl 47; das Sequenzdiagramm in Kapitel 5.7 ist auf die Seitenhöhe begrenzt (es war unten abgeschnitten)	`7774a6d`

Datei	Änderung
`docker-compose.yml`	`NEXT_PUBLIC_API_URL` und `NEXTAUTH_URL` als `${APP_URL:-https://localhost}`
`.env.example`	Eintrag `APP_URL` mit Erläuterung
`scripts/create_user.sh`, `scripts/reset_password.sh`	Lesen `APP_URL` aus der `.env` im Projektstamm
`documentation/Sequenzdiagramm.puml` und `.svg`	Schritte 29 bis 31
`documentation/Schulungsunterlage_Sequenzdiagramm.html`, `.pdf`, PNG zur Adresserfassung	Siehe oben
`documentation/Tutorial_WebApp_v2.5.html` und `.pdf`	Siehe oben
`README.md`, `CLAUDE.md`	`APP_URL` beschrieben; 47 Schritte

Am Code von Backend und Frontend ändert sich nichts, das Datenbankschema bleibt gleich.

Geprüft auf dem Server (Ubuntu, Docker Compose, `APP_URL` auf die Adresse des Servers gesetzt, Zertifikat von mkcert): Start aller vier Container, Umleitung von Port 80 auf 443, Anmeldung, Adressliste und Einstellungen über den Proxy, 401 ohne Token; Import des PLZ-Verzeichnisses (4073 Einträge); Versand der E-Mail «Neue Adresse erfasst» laut Backend-Log. Lokal: `docker compose config` mit und ohne `APP_URL`; alle Codeblöcke des Tutorials zeilenweise gegen die Quelldateien.

Nicht geprüft: die Seiten auf dem Server in einem Browser (nur mit `curl`), die PLZ-Wertehilfe im Formular auf dem Server, der Eingang der E-Mail im Postfach und der Lauf der beiden geänderten Skripte.

Für eine bestehende Installation unter `https://localhost` ist nichts zu tun. Auf einem Server: `APP_URL` in `.env` eintragen, ein Zertifikat für diese Adresse in `nginx/ssl` ablegen, danach `docker compose up -d`.

9. Nachtrag vom 18. Oktober 2026: Technischer Name «bandaddress»

Das Projekt liegt neu im Ordner `bandaddress`, passend zu den anderen Projekten auf demselben Rechner. Alle technischen Namen folgen: Container, Datenbank, Datenbankrolle, .NET-Projekte und Namespaces. Der Anzeigename «Adressverwaltung» in der Oberfläche, in den E-Mails und in den Dokumenten bleibt. Die Versionsnummer bleibt 2.5.0.

Bisher	Neu
Compose-Projekt aus dem Ordnernamen	fest `name: bandaddress` in `docker-`

Bisher	Neu
	`compose.yml`, Volume `bandaddress_postgres-data`
Container `adressverwaltung-db`, `-backend`, `-frontend`, `-nginx`	`bandaddress-db`, `-backend`, `-frontend`, `-nginx`
Datenbank `adressverwaltung`	`bandaddress`
Rolle `adressverwaltung_app`	`bandaddress_app`
`backend/Adressverwaltung.sln`, `AdressverwaltungApi`, `AdressverwaltungApi.Tests` (Ordner, Projekte, Namespaces)	`backend/Bandaddress.sln`, `BandaddressApi`, `BandaddressApi.Tests`
JWT `Issuer`/`Audience` `AdressverwaltungApi`/`AdressverwaltungFrontend`	`BandaddressApi`/`BandaddressFrontend`
Header des Proxys `x-adressverwaltung`	`x-bandaddress`
npm-Paket `adressverwaltung-frontend`	`bandaddress-frontend`

Nachgeführt sind die Hilfsskripte, `db/init/10-app-rolle.sh`, `appsettings.json`, `.env.example`, README, CLAUDE.md, Tutorial, Sicherheitsbericht, beide Schulungsunterlagen (je ein PNG mit dem neuen Header), alle drei Diagramme sowie die zugehörigen SVG und PDF. Die älteren Änderungsprotokolle und die Clean-Code-Analyse beschreiben frühere Stände und bleiben unverändert.

Weil Issuer und Audience ändern, sind alle bisherigen Tokens ungültig: Nach dem Update muss sich jeder neu anmelden.

Geprüft: `dotnet build`, alle 122 Integrationstests, `npm run lint`, `tsc --noEmit`; lokale Migration der bestehenden Daten wie unten beschrieben, danach Start aller vier Container, Login-Seite (200) und 401 ohne Token. Nicht geprüft: die Migration auf dem Server.

Bestehende Installation übernehmen

Die Daten liegen im Volume des alten Compose-Projekts (Name = alter Ordnername, hier `adressverwaltung`). Das alte Volume bleibt als Sicherung stehen.

```
docker compose -p adressverwaltung down
docker volume create --label com.docker.compose.project=bandaddress \
  --label com.docker.compose.volume=postgres-data bandaddress_postgres-data
docker run --rm -v adressverwaltung_postgres-data:/from:ro \
  -v bandaddress_postgres-data:/to alpine cp -a /from/. /to/
docker compose up -d db
docker exec bandaddress-db psql -U postgres -d postgres \
  -c 'ALTER DATABASE adressverwaltung RENAME TO bandaddress' \
  -c 'ALTER ROLE adressverwaltung_app RENAME TO bandaddress_app'
docker compose up --build -d
```

Das Passwort der Rolle bleibt beim Umbenennen erhalten (SCRAM). Läuft alles, kann das alte Volume mit `docker volume rm adressverwaltung_postgres-data` entfernt werden.

Restrisiken und akzeptierte Punkte stehen in `documentation/Sicherheitsbericht.pdf`, Abschnitt 4.